

Java evolution

Module 1 : Java 8 – Fondations modernes.

Module 3 — Java 9 → 11,

Java 9 – principales évolutions

A. Collections immuables (List.of, Set.of, Map.of)

B. Try-with-resources amélioré

C. jshell

Java 10 – type inference avec var

Java 11 – nouvelles APIs et stabilisation LTS

A. Nouveau HttpClient (remplace HttpURLConnection)

B. Méthodes utiles sur String

C. Améliorations diverses

Module 4 — Java 12 → 16

Java 12 – Switch expressions (aperçu)

Java 13–15 – Text Blocks

Java 14–16 – Records (finalisés en 16)

Module 5 – Java 17 (LTS)

Sealed Classes – Contrôler l'héritage

Variante mixte

Pattern Matching pour switch (aperçu en 17)

Combinaison Records + Pattern Matching

Autres améliorations (Java 17)

Module 6 : Java 21 Advanced Features (LTS)

Virtual Threads — Project Loom

Sequenced Collections

Record Patterns + Pattern Matching for Switch (finalized)

Scoped Values

String Templates

Module 1 : Java 8 – Fondations modernes.

Avant Java 8, Java était purement orienté objet ; à partir de Java 8, il devient **hybride** : orienté objet **et** fonctionnel.

Nouveautés majeures:

Domaine	Fonctionnalité	Description rapide
Fonctions anonymes	Lambdas	Permettent d'écrire du code plus concis et expressif

Collections	Stream API	Chaînage d'opérations (filter, map, reduce...)
Null-safety	Optional	Conteneur pour éviter les <code>NullPointerException</code>
Dates	java.time	Nouvelle API de date/heure (remplace <code>Date</code> et <code>Calendar</code>)
Concurrence	CompletableFuture	Exécution asynchrone simplifiée
Interfaces	Méthodes par défaut	Permettent d'ajouter une méthode dans une interface sans casser les implémentations existantes

```

1 CompletableFuture.supplyAsync(() -> fetchData())
2     .thenApply(data -> process(data))
3     .thenAccept(System.out::println);

```

⇒ Asynchrone sans `Thread` explicite.

Méthodes par défaut dans les interface:

```

1 interface Vehicle {
2     void move();
3     default void stop() {
4         System.out.println("Vehicle stopped");
5     }
6 }

```

Module 3 — Java 9 → 11,

Java 9 – principales évolutions

A. Collections immuables (`List.of` , `Set.of` , `Map.of`)

Avant Java 9 :

```

1 List<String> list = new ArrayList<>();
2 list.add("A");
3 list.add("B");
4

```

Depuis Java 9 :

```
1 List<String> list = List.of("A", "B"); // immutable
2 Set<Integer> set = Set.of(1, 2, 3);
3 Map<Integer, String> map = Map.of(1, "A", 2, "B");
4
```

Tentative de `add` ou `remove` ⇒ `UnsupportedOperationException`.

B. Try-with-resources amélioré

Plus besoin de redéclarer la ressource :

```
1 BufferedReader br = new BufferedReader(new FileReader("file.txt"));
2 try (br) { // <- variable déjà existante
3     br.lines().forEach(System.out::println);
4 }
5
```

c. jshell

```
1 jshell> int x = 5;
2 jshell> x + 2
```

Parfait pour tester du code rapide.

D. Module System (JPMS)

Nécessaire pour très gros projets, mais à connaître :

- `module-info.java`
- contrôle explicite des dépendances / export de packages

Pour l'instant, retiens : c'est optionnel, mais utile pour des applis cloisonnées.

Java 10 – type inference avec `var`

Permet d'alléger le code **sans perdre le typage statique** :

```
1 var list = List.of("A", "B", "C"); // List<String>
2 for (var item : list) {
3     System.out.println(item);
4 }
```

`var` ne peut être utilisé que pour des **variables locales initialisées**.

Pas pour : champs, paramètres, ou valeurs non-initialisées.

Java 11 – nouvelles APIs et stabilisation LTS

A. Nouveau `HttpClient` (remplace `URLConnection`)

Exemple synchrone :

```
1 import java.net.http.*;
2 import java.net.URI;
```

```

3
4 HttpClient client = HttpClient.newHttpClient();
5 HttpRequest request = HttpRequest.newBuilder()
6     .uri(URI.create("https://api.github.com"))
7     .build();
8
9 HttpResponse<String> response =
10     client.send(request, HttpResponse.BodyHandlers.ofString());
11
12 System.out.println(response.statusCode());
13 System.out.println(response.body());
14

```

Asynchrone :

```

1 client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
2     .thenApply(HttpResponse::body)
3     .thenAccept(System.out::println);
4

```

B. Méthodes utiles sur String

```

1 "   ".isBlank();           // true
2 "abc\n".lines().count();   // 1
3 "Hello".repeat(3);         // HelloHelloHello

```

C. Améliorations diverses

- Fichiers en ligne de commande : `java Hello.java`
- Garbage Collector G1 par défaut
- Support complet TLS 1.3

Catégorie	Java 8	Java 9–11 ajout
Lambdas	✓	—
Streams	✓	—
Optional	✓	—
API Date	✓	—
Collections immuables	✗	✓
<code>var</code>	✗	✓
HttpClient moderne	✗	✓
String utils (<code>isBlank</code> , <code>repeat</code>)	✗	✓
JShell	✗	✓
Module System	✗	✓

Module 4 — Java 12 → 16

Java 12 – Switch expressions (aperçu)

Avant :

```
1 int num = 2;
2 String result;
3 switch (num) {
4     case 1: result = "un"; break;
5     case 2: result = "deux"; break;
6     default: result = "autre";
7 }
8
```

Depuis Java 14 (finalisé en Java 14) :

```
1 int num = 2;
2 String result = switch (num) {
3     case 1 -> "un";
4     case 2 -> "deux";
5     default -> "autre";
6 };
7
```

Plus de `break` , plus clair, retourne directement une valeur.

Java 13–15 – Text Blocks

Les **text blocks** simplifient les chaînes multilignes :

```
1 String json = ""
2 {
3     "name": "Mouad",
4     "city": "Casablanca"
5 }
6 "";
7
8 System.out.println(json);
9
```

Fini les `"\n"` et les concaténations.

Java 15 a finalisé cette feature.

Java 14–16 – Records (finalisés en 16)

Un record est une classe immuable, idéale pour DTO / valeurs simples :

Avant :

```
1 class Person {
2     private final String name;
3     private final int age;
4
5     public Person(String name, int age) {
6         this.name = name;
7         this.age = age;
8     }
9 }
```

```

8     }
9
10    public String name() { return name; }
11    public int age() { return age; }
12 }

```

Depuis Java 16 :

```

1 public record Person(String name, int age) {}

```

Automatiquement :

- champs `final`
- constructeur
- getters (`name()` , `age()`)
- `equals` , `hashCode` , `toString`

Module 5 – Java 17 (LTS)

Découvrir ce que **Java 17** apporte de stable :

- **Sealed classes** : hiérarchies contrôlées
- **Pattern matching** pour `switch` (aperçu)
- **Records + Pattern matching** combinés
- **Améliorations API & performance**

Sealed Classes – Contrôler l'héritage

Tu définis quelles classes ont le droit d'hériter d'une super-classe.

Cela rend les hiérarchies plus sûres et plus explicites.

```

1 public abstract sealed class Shape
2     permits Circle, Rectangle, Triangle { }
3
4 public final class Circle extends Shape { }
5 public final class Rectangle extends Shape { }
6 public final class Triangle extends Shape { }
7

```

Seule une classe listée dans `permits` peut hériter de `Shape` .

Chaque sous-classe doit être `final` , `sealed` , ou `non-sealed` .

Variante mixte

```

1 public abstract sealed class Vehicle permits Car, Truck, Drone { }
2
3 public final class Car extends Vehicle { }
4 public sealed class Truck extends Vehicle permits ElectricTruck { }
5 public non-sealed class Drone extends Vehicle { } // libre héritage
6

```

Lisibilité + sécurité : impossible d'ajouter un type inattendu ailleurs.

Pattern Matching pour `switch` (aperçu en 17)

On peut combiner `instanceof` + `switch` de façon élégante :

```
1 static String describe(Object obj) {  
2     return switch (obj) {  
3         case Integer i -> "Integer: " + (i * 2);  
4         case String s  -> "String: " + s.toUpperCase();  
5         case null      -> "Null value";  
6         default        -> "Unknown type";  
7     };  
8 }  
9
```

Ce pattern matching pour `switch` était en **aperçu** dans Java 17, finalisé en 21.

Combinaison Records + Pattern Matching

Tu peux déstructurer un `record` dans un `switch` :

```
1 public record Point(int x, int y) {}  
2  
3 static String position(Point p) {  
4     return switch (p) {  
5         case Point(int x, int y) when x == 0 && y == 0 -> "Origine";  
6         case Point(int x, int y) when x == 0 -> "Sur axe Y";  
7         case Point(int x, int y) when y == 0 -> "Sur axe X";  
8         default -> "Autre point";  
9     };  
10 }
```

Autres améliorations (Java 17)

- **RandomGenerator** API moderne :

```
RandomGeneratorFactory.of("L64X128MixRandom")
```

- **Suppressions** : Applet API, RMI Activation, expérimental Nashorn JS engine
- **Nouvel algorithme de GC** : ZGC et G1 stabilisés
- **Sécurité accrue** : interdiction de strong encapsulation par défaut (`--illegal-access` supprimé)

Module 6 : Java 21 Advanced Features (LTS)

Master the big innovations introduced or finalized in **Java 21 LTS**:

- **Virtual Threads** (JEP 444)
- **Sequenced Collections** (JEP 431)
- **Record Patterns & Pattern Matching for Switch** (final in 21)
- **Scoped Values** (JEP 446 preview)
- **String Templates** (JEP 430 preview)

A **Virtual Thread** is a lightweight, OS-independent thread managed by the JVM.

You can create **thousands** of them cheaply for blocking I/O tasks.

```
1 public class VirtualThreadsDemo {
2     public static void main(String[] args) throws InterruptedException
3     {
4         try (var executor =
5             java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
6             for (int i = 0; i < 10; i++) {
7                 int id = i;
8                 executor.submit(() -> {
9                     System.out.println("Task " + id + " → " +
10                        Thread.currentThread());
11                     Thread.sleep(200);
12                     return null;
13                });
14            }
15        }
16    }
17 }
```

Each task runs in its own virtual thread — minimal memory and context-switch cost.

Use them whenever you do **blocking I/O (HTTP, DB, file)** instead of classic thread-pools.

Sequenced Collections

Finally, `List`, `Set`, and `Map` now share common order-aware methods:

```
1 List<String> list = List.of("A", "B", "C");
2 System.out.println(list.getFirst()); // A
3 System.out.println(list.getLast());  // C
4 list.reversed().forEach(System.out::println); // C, B, A
5
6 Map<Integer, String> map = Map.of(1, "one", 2, "two");
7 System.out.println(map.firstEntry()); // 1=one
8 System.out.println(map.lastEntry());  // 2=two
9
```

Consistent iteration order APIs across all collections.

Record Patterns + Pattern Matching for Switch (finalized)

You already used these in the last module — Java 21 finalized them (no `--enable-preview`).

```
1 record Point(int x, int y) {}
2
3 static String describe(Object o) {
4     return switch (o) {
5         case Point(int x, int y) when x == y -> "Diagonal";
6         case Point(int x, int y) -> "Point (" + x + ", " + y + ")";
7         default -> "Unknown";
8     };
9 }
10
```

Scoped Values

A safer, immutable replacement for `ThreadLocal` — lets you share read-only context across threads (even virtual threads).

```
1 import jdk.incubator.concurrent.ScopedValue;
2
3 public class ScopedValuesDemo {
4     static final ScopedValue<String> USER = ScopedValue.newInstance();
5
6     public static void main(String[] args) {
7         ScopedValue.where(USER, "Mouad")
8             .run(() -> {
9                 System.out.println("Current user: " +
10                     USER.get());
11             });
12     }
13 }
```

Perfect for per-request context (auth, trace IDs) without leaking memory between threads.

String Templates

Simpler, safer string interpolation (still preview):

```
1 String name = "Mouad";
2 int age = 26;
3
4 String message = STR."Hello \{name}, you are \{age} years old!";
5 System.out.println(message);
6
```

→ No `String.format`, no concatenation, and injection-safe.